# Laboratory Exercise 4: Numerical Computing with NumPy & Text Processing

| Student Details | |
|---|---|
| **Student Name:** _____ | **Student ID:** _____ |
| **Duration:** 2 Hours (120 Minutes) | **Total Points:** 100 Points |

## 1. Learning Objectives

- **Text Processing & Formatting:** Format telemetry strings using $f$-strings ( `.2f` , `.2e` , alignment) and parse raw text with `.strip()` and `.split()` .
- **NumPy Array Fundamentals:** Create 1-D/2-D arrays, inspect dimensions ( `.shape` , `.ndim` , `.dtype` ), and generate arrays ( `linspace` , `zeros` , `eye` ).
- **Vectorization & Grid Analysis:** Apply broadcasting, boolean masking, and axis reductions ( `axis=0` , `axis=1` ) to analyze 2-D elevation matrices.

## 2. Exercise Structure, Evaluation & Instructor Sign-Off

This 120-minute laboratory session is divided into 5 sequential parts totaling 100 points: **Part 1: String Formatting, Specifiers & Text Parsing** (20 mins, 15 pts), **Part 2: NumPy Array Creation & Dimensionality Inspection** (25 mins, 20 pts), **Part 3: Vectorization & Broadcasting vs. Traditional Loops** (25 mins, 20 pts), **Part 4: Practical Modeling: Airport Runway Surface Elevation Matrix** (30 mins, 25 pts), and **Part 5: Challenge: Raw Telemetry Stream Parser & Grid Statistics** (20 mins, 20 pts).

| Instructor Verification | Details |
|---|---|
| **Score Awarded:** _____ / 100 Points | **Instructor Signature:** _____ **Date:** _____ / _____ / 20__ |
| **Instructor Comments & Feedback:** | _____ |

## Part 1: String Formatting, Specifiers & Text Parsing (15 Points)

### Task 1.1: Precision & Alignment Formatting (8 Points)

Given the following navigation variables:

```python
transponder_code = 7500
ground_speed = 432.8456
heading = 89.2
distance_km = 1425600.0  # in meters
```

Write Python print statements using **f-strings** to output:

1. `transponder_code` zero-padded to 4 digits: e.g., `"7500"` (or `"0750"` for code 750).

   **Output / Formatted String:** _____

2. `ground_speed` rounded to 1 decimal place with a field width of 8 characters, right-aligned: `f"{ground_speed:>8.1f}"`.

   **Output / Formatted String:** _____

3. `distance_km` in scientific notation with 2 decimal places: `f"{distance_km:.2e}"`.

   **Output / Formatted String:** _____

### Task 1.2: String Sanitation & Tokenization (7 Points)

A raw METAR (meteorological aerodrome report) line is received over a radio data link with unwanted whitespace and punctuation:

```python
raw_metar = "  VTBS 040900Z 23012KT 9999 FEW025 32/24 Q1008 NOSIG  \n"
```

Write code to:

1. Strip leading and trailing whitespace using `.strip()`.
2. Split the string into distinct token words using `.split()`.
3. Extract and display:
   - The ICAO airport code ( `"VTBS"` )
   - The wind observation ( `"23012KT"` )
   - The QNH altimeter setting ( `"Q1008"` )

**Student Answer / Program Output:**

- **Stripped & Split Token List:** _____
- **ICAO Airport Code:** _____
- **Wind Observation:** _____

- **QNH Altimeter Setting:** _____

---

## Part 2: NumPy Array Creation & Dimensionality (20 Points)

### Task 2.1: 1-D vs. 2-D Structural Inspection (10 Points)

Import NumPy as `np`. Create the following two arrays:

```python
import numpy as np

arr_1d = np.array([120, 135, 140, 155, 160])
arr_2d = np.array([[120, 135, 140, 155, 160]])
```

1. **(4 Points)** Print the `.ndim`, `.shape`, and `.dtype` for both arrays.

   **`arr_1d` Inspection:** `.ndim` = _____ | `.shape` = _____ | `.dtype` = _____ **`arr_2d` Inspection:** `.ndim` = _____ | `.shape` = _____ | `.dtype` = _____

2. **(6 Points)** Explain in your own words (in a comment) why `arr_1d.shape` returns `(5,)` while `arr_2d.shape` returns `(1, 5)`, and what difference this makes when performing matrix algebra.

   **Student Explanation:**

   _____
   _____
   _____

### Task 2.2: Array Generation Helpers (10 Points)

Use NumPy built-in helper functions to generate:

1. A 1-D array of 8 equidistant calibration test frequencies between $118.0$ MHz and $136.0$ MHz (use `np.linspace`).

   **Array Output:** _____

2. A $3 \times 4$ 2-D array of zeros with data type `float64` (use `np.zeros`).

   **Shape:** `(____, ____)` | **Dtype:** _____

3. A $4 \times 4$ identity matrix representing an aircraft rotation state space (use `np.eye`).

   **Shape:** `(____, ____)` | **Diagonal Elements:** _____

---

## Part 3: Vectorization & Broadcasting vs. Traditional Loops (20 Points)

### Task 3.1: Unit Conversion Benchmark (10 Points)

Air traffic computers frequently convert indicated airspeeds from knots to meters per second ( $1\ \text{knot} = 0.514444\ \text{m/s}$):

```python
speeds_knots = np.array([110.0, 135.5, 160.0, 220.0, 250.0, 310.0, 420.0, 480.0])
```

1. Convert `speeds_knots` into `speeds_mps` using **vectorized multiplication** (broadcasting a scalar without writing any loop).
2. Display both arrays side-by-side using a loop with `zip()` formatted to 2 decimal places.

**Student Answer / Program Output:**

- **Vectorized Expression:** _____
- **Sample Conversions (knots $\rightarrow$ m/s):**
  - 110.0 kt = _____ m/s | 250.0 kt = _____ m/s | 480.0 kt = _____ m/s

### Task 3.2: Boolean Filtering & Array Masking (10 Points)

Using `speeds_knots` from Task 3.1:

1. Create a boolean mask identifying all speeds exceeding $250.0\ \text{knots}$ (terminal airspace speed limit): `mask = speeds_knots > 250.0` .
2. Use the mask to index into `speeds_knots` and extract only the compliant high-speed cruise values.
3. Compute the percentage of recorded speeds that violated the $250.0\ \text{knot}$ restriction.

**Student Answer / Program Output:**

- **Boolean Mask Result:** `[_____]`
- **Extracted High-Speed Values (> 250 kt):** `[_____]`
- **Violation Percentage:** _____%

---

## Part 4: Practical Modeling: Runway Elevation Matrix Analysis (25 Points)

A LIDAR survey scans the surface elevation (in meters above sea level) across a $4 \times 6$ grid along Runway 19L:

```python
runway_grid = np.array([
    [14.2, 14.3, 14.5, 14.4, 14.2, 14.1],
    [14.3, 14.5, 14.8, 14.7, 14.4, 14.2],
    [14.4, 14.6, 14.9, 14.8, 14.5, 14.3],
    [14.1, 14.2, 14.4, 14.3, 14.1, 13.9]
])
```

### Requirements:

Write a Python program that performs the following multi-dimensional NumPy computations:

1. **Dimensions:** Print the array dimensions ( `.shape` ) and total sensor points ( `.size` ).
2. **Global Extremes:** Find and display the overall minimum and maximum elevation across the entire runway.
3. **Cross-Section Average (Axis = 0):** Compute the average elevation along each of the 6 longitudinal stations (down each column).
4. **Lateral Section Average (Axis = 1):** Compute the average elevation across each of the 4 lateral profiles (across each row).
5. **Runway Slope & Hazard Check:**
   - Compute the total longitudinal elevation drop:

$$\Delta h = \text{Mean of Station } 0 - \text{Mean of Station } 5$$

   - If any individual sensor point exceeds $14.85\ \text{m}$, use `np.any()` or `np.where()` to flag a `"[HIGH RIDGE HAZARD]"` alert.

**Example Run:**

```
text
```

```
Runway Grid Shape: (_, _) | Total Survey Points: __
Minimum Elevation: ____ m
Maximum Elevation: ____ m
Longitudinal Station Means (Cols): [_____________________________________] m
Lateral Profile Means (Rows): [_____________________________] m
Longitudinal Gradient (Station 0 to 5): _______ m
Safety Inspection: ____________________________________
```

**Student Answer / Program Output:**

- **Runway Grid Dimensions:** Shape = `(____, ____)` | Total Survey Points = _____
- **Elevation Range:** Min = _____ m | Max = _____ m
- **Longitudinal Means (Cols, Axis 0):** `[_____________________________________]` m
- **Lateral Profile Means (Rows, Axis 1):** `[_____________________________________]` m
- **Longitudinal Gradient ($\Delta h$):** _____ m
- **Hazard Inspection Status:** _____

## Part 5: Challenge Task — Raw Telemetry Stream Parser & Sensor Grid (20 Points)

A ground radar telemetry receiver outputs a continuous formatted data stream as comma-separated text strings for 3 aircraft targets:

```
python
```

```
telemetry_log = [
    "AC_ID:THY401,ALT:32000,SPD:450.5,LAT:13.68,LON:100.75",
    "AC_ID:SIA122,ALT:38000,SPD:482.0,LAT:13.72,LON:100.82",
```

```
    "AC_ID:BAW009,ALT:29000,SPD:420.2,LAT:13.60,LON:100.68"
]
```

## Requirements:

Write a Python program that:

1. Iterates through `telemetry_log`, splits each string, and extracts the numeric values for `ALT` (Altitude) and `SPD` (Ground Speed).
2. Packages the extracted data into a $3 \times 2$ 2-D NumPy array where column 0 is Altitude and column 1 is Speed.
3. Computes:
   - Average altitude across all targets.
   - Maximum speed observed and the associated aircraft ID.
4. Uses an $f$-string table to print a clean air situation display report.

**Example Run:**

```
text
```

```
--- Active Radar Telemetry Summary ---
Callsign   | Altitude (ft) | Speed (kts)
----------------------------------------
THY401     |    32,000     |    450.50
SIA122     |    38,000     |    482.00
BAW009     |    29,000     |    420.20
----------------------------------------
Mean Radar Altitude: 33,000.00 ft
Fastest Track: SIA122 at 482.00 kts
```

**Student Answer / Program Output:**

- **Extracted 2-D NumPy Array Shape:** `(____, ____)`
- **Mean Radar Altitude:** _____ ft
- **Fastest Track:** Callsign = _____ | Speed = _____ kts